



# Desenvolvimento de Sistemas para Web

## Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br



# Introdução ao Python

## Resumo

Python é uma linguagem de programação poderosa de uso geral. Pode ser usada de forma interativa, permitindo um desenvolvimento muito rápido. Possui muitas ferramentas de computação científica poderosas, tornando-a ideal para matemática aplicada e computacional. Nesta aula faremos uma revisão focada dos principais conceitos de Python que serão mais utilizados ao longo do curso. Abordaremos variáveis, tipos de dados, operadores, arquivos e execução, funções (incluindo parâmetros default, \*args, \*\*kwargs, type hints e docstrings), o bloco `if __name__ == "__main__"`, decorators, classes, herança e organização em módulos/pacotes. O objetivo é reduzir erros comuns que surgem ao começar a desenvolver aplicações em Python.

*“A mente que se abre a uma nova ideia jamais volta ao seu tamanho original.”*

Albert Einstein

## Vamos começar

Python é uma linguagem multiparadigma, com forte suporte à Programação Orientada a Objetos (POO). Um **objeto** é uma instância de uma classe, com **atributos** (dados) e **métodos** (comportamentos) [1].

Python ganhou relevância acelerada em computação científica, análise de dados, automação e prototipagem rápida [2]. Os erros mais frequentes de iniciantes incluem:

- má compreensão de escopo de variáveis
- confusão entre listas, tuplas e dicionários
- uso incorreto de funções e argumentos
- problemas de indentação
- não entender o bloco `if __name__ == "__main__"`
- dificuldade inicial com decorators e orientação a objetos

Por isso, antes de iniciar os desenvolvimentos, vamos reforçar esses fundamentos.

## Variáveis — como criar, usar e entender os tipos

Em Python, criar uma variável é extremamente simples: escolha um nome e atribua um valor usando um único sinal de igual `=`.

- `=` (um sinal só) → atribuição: “este nome agora guarda este valor”
- `==` (dois sinais) → comparação: “estes dois valores são iguais?” (responde `True` ou `False`)

Exemplos rápidos para fixar a diferença:

```
1 # Atribuicao (criando ou alterando o valor)
2 idade = 25
3 nome = "Joao"
4 altura = 1.78
5 eh_estudante = True
6
7 # Comparacao (perguntando)
8 idade == 25          # True
9 idade == 30          # False
10 nome == "Maria"     # False
```

### NOTA

**Dica importante que evita muita confusão no começo:**

Muitos iniciantes trocam `=` por `==` dentro de um `if` ou `while`. Isso gera um

erro claro do Python (geralmente `SyntaxError` ou `TypeError`).

Lembre-se sempre:

- `x = 10` → “estou colocando 10 dentro de x”
- `x == 10` → “estou perguntando se x vale 10” (resultado: `True` ou `False`)

Python é uma linguagem de **tipagem dinâmica**: você não precisa informar o tipo da variável ao criá-la. O interpretador descobre sozinho.

```
1 # Exemplos de diferentes tipos sem declarar nada
2 idade = 25                # inteiro (int)
3 altura = 1.75             # decimal (float)
4 nome = "Mariana"          # texto (str)
5 ativo = True              # booleano (bool)
6 notas = [8.5, 7.0, 9.2]   # lista
7
8 print(type(idade))        # <class 'int'>
9 print(type(altura))       # <class 'float'>
10 print(type(nome))        # <class 'str'>
11 print(type(ativo))       # <class 'bool'>
12 print(type(notas))       # <class 'list'>
13
14 # E o tipo pode mudar durante o programa!
15 idade = "vinte e cinco"   # agora virou str
16 print(type(idade))       # <class 'str'>
```

Isso deixa o código mais rápido de escrever e flexível, mas exige cuidado: o tipo pode mudar inesperadamente se você não prestar atenção.

## Atribuição múltipla – prática e muito usada

Você pode atribuir valores a várias variáveis ao mesmo tempo. Essa técnica aparece bastante em códigos reais, especialmente em loops, funções e leitura de dados.

```
1 # Forma tradicional (uma por linha)
2 a = 5
3 b = 10
```

```
4
5 # Forma curta (atribuicao multipla)
6 a, b = 5, 10
7
8 # Troca de valores sem variavel auxiliar (muito comum!)
9 a, b = b, a          # agora a = 10 e b = 5
10 print(a, b)         # 10 5
```

Exemplo prático que você verá com frequência:

```
1 nome, idade, cidade = "Ana", 22, "Serra"
2 print(f"{nome} tem {idade} anos e mora em {cidade}.")
3 # Resultado: Ana tem 22 anos e mora em Serra.
```

## Problema

**A.1** Crie três variáveis: produto, preco e quantidade. Atribua valores de exemplo (por exemplo: "mouse", 89.90, 5). Em seguida, use `print()` com f-string para mostrar algo como: "O produto mouse custa R\$89.90 e temos 5 unidades em estoque."

**A.2** Tente prever o resultado das seguintes comparações (responda mentalmente antes de executar):

- (a) `10 == 10`
- (b) `"python" == "Python"`
- (c) `3.0 == 3`
- (d) `True == 1`
- (e) `True == "True"`

**A.3** Escreva uma única linha que troque os valores das variáveis `x` e `y` sem usar uma terceira variável auxiliar.

**A.4** (Bônus) Crie uma variável `usuario` que receba, em uma única linha de atribuição

múltipla, os valores "Wanderson", 30, "Serra". Depois imprima uma frase usando f-string.

## Operadores aritméticos e ordem de precedência

Python possui os operadores matemáticos mais comuns. A maioria deles funciona exatamente como na matemática do ensino médio.

- + soma
- - subtração
- \* multiplicação
- / divisão real (sempre retorna número com casas decimais — float)
- // divisão inteira (arredonda para baixo — retorna int ou float dependendo dos operandos)
- \% resto da divisão inteira (módulo)
- \*\* potenciação / exponenciação

### Código: Exemplos básicos de operadores

```
1 # Divisao comum (sempre float)
2 print(100 / 7)          # 14.285714285714286
3
4 # Divisao inteira (arredonda para baixo)
5 print(100 // 7)         # 14
6 print(101 // 7)         # 14
7 print(-101 // 7)        # -15   (atencao: arredonda para baixo, nao trunca)
8
9 # Resto da divisao
10 print(100 % 7)          # 2    --> 7*14 = 98 --> sobra 2
11 print(25 % 4)           # 1
12
13 # Potenciacao
14 print(2 ** 10)          # 1024
15 print(3 ** 3)           # 27
```

## NOTA

**Ordem de precedência (o que o Python calcula primeiro):**

1. Parênteses ( )
2. Exponenciação \*\*
3. Multiplicação, divisão, divisão inteira e resto \* / // % (da esquerda para a direita)
4. Adição e subtração + - (da esquerda para a direita)

**Exemplo prático:**

```
1 print(10 + 3 * 4 ** 2)
2 # Passo a passo:
3 # 1. 4 ** 2 --> 16
4 # 2. 3 * 16 --> 48
5 # 3. 10 + 48 --> 58
```

Resultado: **58**

## Tipagem explícita com type hints (opcional, mas muito recomendada)

Já discutimos que Python é dinamicamente tipado — o interpretador descobre os tipos em tempo de execução. Porém, desde o Python 3.5 (e com melhorias constantes), podemos deixar os tipos **explícitos** usando *type hints* (anotações de tipo).

Elas são opcionais, não mudam o comportamento do código em runtime, mas trazem vantagens enormes:

- Código muito mais legível e auto-documentado.
- Ferramentas como mypy, pyright e Ruff encontram erros de tipo antes de você rodar o programa.
- Excelente suporte em editores (autocompletar, refatoração segura).
- Facilita manutenção em projetos Flask/Django maiores.

A sintaxe é simples:

```
1 # Variavel com tipo explicito
2 nome: str = "Wanderson"
3 idade: int = 30
4 print(nome, idade)
```

## NOTA

### Onde aprender mais?

O sistema de type hints foi introduzido pela **PEP 484 – Type Hints** (2014), escrita por Guido van Rossum e outros. Ela é a principal referência e está finalizada/implementada. Link: <https://peps.python.org/pep-0484/>

Outras PEPs complementares importantes:

- PEP 526: anotações em variáveis avulsas
- PEP 585: `list[str]` sem importar do typing
- PEP 604: união com `|` (ex: `int | None`)

Recomendação prática: comece usando type hints em funções novas. Instale o mypy (`pip install mypy`) e rode `mypy seu_arquivo.py` para ver os benefícios imediatamente!

## Problema

**A.5** Sem usar calculadora, qual é o resultado de  $5 + 4 * 3 ** 2$  ? Confirme executando no Python.

**A.6** Qual o resultado de  $50 // 6$  e  $50 \% 6$  ? O que cada um significa na prática?

**A.7** Escreva uma expressão que use parênteses para forçar que a soma seja feita antes da multiplicação no exemplo acima.

## Como criar e executar seu primeiro arquivo Python

Todo código Python que você quiser salvar e executar depois deve ser gravado em um arquivo com a extensão `.py`. Exemplos de nomes válidos: `ola.py`, `meu_programa.py`, `calculadora.py`.

Para quem está começando, a recomendação é simples: **use um editor de texto leve e direto**. Boas opções gratuitas incluem:

- **VS Code** (muito popular, leve e com ótima extensão Python)
- **Gedit** (simples, já vem no Linux)
- **Notepad++** (leve, bom para Windows)
- **Sublime Text** ou **Vim/Nano** (se você gosta de minimalismo extremo)

Evite, no início, instalar IDEs pesadas como PyCharm ou Spyder. Elas têm muitos recursos avançados que podem distrair e confundir quem ainda está aprendendo a sintaxe básica. Um editor simples ajuda você a **realmente entender** o que está acontecendo.

### Exemplo de um arquivo Python bem básico

Crie um arquivo chamado `ola.py` e coloque o seguinte conteúdo:

**Código:** `ola.py`

```
1 # Isto eh um comentario de uma linha so.
2 # Tudo depois do # eh ignorado pelo Python.
3
4 """
5 Aqui voce pode escrever
6 varias linhas de texto.
7 Isso eh uma string de multiplas linhas.
8 Muito usado para colocar explicacoes longas
9 ou documentacao no inicio dos arquivos.
10 """
```

```
11
12 if __name__ == "__main__":
13     pass          # 'pass' significa: "nao faca nada por enquanto"
14                 # Eh como um espaco reservado temporario.
```

Voltaremos à instrução `if __name__ == "__main__"` na seção de funções.

## Comentários em Python — por que eles são importantes?

Existem duas formas principais de colocar comentários:

1. Usando `\#` → comentário de uma única linha
2. Usando três aspas (`"""` ou `'''`) → pode ocupar várias linhas

Exemplo de comentário de várias linhas:

```
1 """
2 Esta funcao calcula o quadrado de um numero.
3 Ela sera usada no exercicio 3.
4 """
```

**Dica importante:** As strings de três aspas no topo do arquivo (ou logo abaixo de uma função) costumam ser usadas como **docstrings** — elas viram documentação automática quando alguém usa a função `help()` ou ferramentas como Sphinx / makedocs. Docstrings completas serão vistas na seção de funções.

---

## Problema

**A.8** Abra seu editor e crie um novo arquivo chamado `python_intro.py`.

**A.9** Cole exatamente este código:

**Código:** `python_intro.py`

```
1 if __name__ == "__main__":  
2     print("Ola, mundo!")      # Use 4 espacos para indentar (NUNCA misture  
                                tab e espacos!)
```

**A.10** Salve o arquivo em uma pasta fácil de encontrar (exemplo: dentro de Documentos ou Área de Trabalho).

**A.11** Abra o terminal / prompt de comando:

- No Linux ou macOS: abra o *Terminal*
- No Windows: abra o *Prompt de Comando* ou *Git Bash* (recomendado)

**A.12** Navegue até a pasta onde salvou o arquivo. Use estes comandos básicos:

```
# Ver em qual pasta voce esta agora  
pwd  
  
# Listar arquivos e pastas da pasta atual  
ls          # no Linux/macOS  
dir         # no Windows Prompt  
  
# Entrar em uma pasta (exemplo)  
cd Documentos  
cd python-aulas      # se voce criou uma subpasta  
  
# Verificar se o arquivo esta la  
ls  
# Deve aparecer: python_intro.py
```

**A.13** Execute o programa:

```
python python_intro.py
```

**A.14** Se tudo der certo, você verá na tela:

```
Olá, mundo!
```

Parabéns! Esse é o seu primeiro programa Python executado a partir de um arquivo.

## Funções – o coração do código reutilizável

Funções são blocos de código reutilizáveis que recebem entradas (parâmetros/argumentos), executam uma tarefa e (geralmente) devolvem um resultado.

### Definindo uma função básica

Para criar uma função, use a palavra-chave `def`, seguida do nome da função, parênteses com os parâmetros e dois pontos. O corpo da função deve ser indentado com **exatamente quatro espaços**.

**Código:** Exemplo simples

```
1 def add(x, y):  
2     """Soma dois numeros."""  
3     return x + y
```

Chamada:

```
1 resultado = add(3, 7)    # resultado = 10
```

#### NOTA

##### Python é obcecado por indentação!

Diferente de linguagens como C, Java ou JavaScript (que usam chaves {}), Python usa a **indentação** como parte da sintaxe. Isso torna o código mais limpo e legível, mas exige disciplina.

Regras de ouro (de acordo com a **PEP 8 – Style Guide for Python Code**):

- Use sempre **4 espaços** por nível de indentação
- **Nunca misture espaços e tabulações** no mesmo arquivo — isso causa erros difíceis de achar (`IndentationError` ou `TabError`)
- Configure seu editor para que a tecla Tab insira 4 espaços (*soft tabs*)

Link oficial da PEP 8: <https://peps.python.org/pep-0008/> (seção sobre Indentation).

## Parâmetros vs argumentos

- **Parâmetros:** nomes que aparecem na definição da função (ex: `width`, `height`)
- **Argumentos:** valores reais passados quando você chama a função (ex: `2`, `5`)

Exemplo:

```
1 def area(width, height):
2     return width * height
3
4 print(area(2, 5))      # 10  <-- 2 e 5 sao argumentos
```

## Retorno de valores (incluindo múltiplos)

A instrução `return` encerra a função e devolve um valor (ou vários).

```
1 def arithmetic(a, b):
2     """Retorna diferenca e produto."""
3     return a - b, a * b
4
5 diff, prod = arithmetic(5, 2)
6 print(diff, prod)      # 3 10
```

Se não houver `return`, a função retorna implicitamente `None` (equivalente a `null` em outras linguagens).

```
1 def oops(i):  
2     print(i + 1)      # imprime, mas nao retorna  
3  
4 x = oops(1999)        # x recebe None  
5 print(x)              # None
```

#### NOTA

**Dica crucial:** Se você quer usar o resultado depois, **sempre use return**. Chamar `print()` dentro da função não faz ela “retornar” nada!

## Valores padrão para parâmetros

Você pode definir valores padrão — se o argumento não for passado, usa o valor padrão.

```
1 def saudacao(nome, horario="dia"):  
2     """Retorna uma saudacao personalizada.  
3     Args:  
4         nome (str): Nome da pessoa  
5         horario (str): Momento do dia (opcional)  
6     Returns:  
7         str: Mensagem de saudacao  
8     """  
9     return f"Bom {horario}, {nome}!"  
10  
11 print(saudacao("Mariana"))      # Bom dia, Mariana!  
12 print(saudacao("Pedro", "noite")) # Bom noite, Pedro!
```

Regras importantes:

- Parâmetros sem default devem vir **antes** dos com default na definição
- Na chamada: argumentos posicionais primeiro, depois nomeados (keyword)

Exemplo de erro comum:

```

1 # Errado:
2 def errado(c=0, a, b): ... # SyntaxError: non-default argument follows default
3
4 # Correto:
5 def pad(a, b, c=0):
6     print(a, b, c)
7
8 pad(1, 2)          # 1 2 0
9 pad(1, 2, 3)       # 1 2 3
10 pad(b=4, a=5)      # 5 4 0  <-- ordem nao importa se usar nomes
11 pad(7, c=9, b=8)   # 7 8 9

```

## Argumentos arbitrários: \*args e \*\*kwargs

Útil quando você não sabe quantos argumentos virão (muito usado no desenvolvimento de extensões).

```

1 def criar_usuario(**dados):
2     """Cria usuario com dados arbitrarios."""
3     print("Novo usuario criado com os dados:", dados)
4
5 criar_usuario(nome="Luiza", idade=29, cidade="Serra", curso="Engenharia")
6 # {'nome': 'Luiza', 'idade': 29, 'cidade': 'Serra', 'curso': 'Engenharia'}

```

\*args recebe argumentos posicionais como tupla; \*\*kwargs recebe nomeados como dicionário.

## Type hints em funções (recomendado em projetos reais)

Desde Python 3.5 (PEP 484), você pode anotar tipos explicitamente. Não afeta execução, mas melhora legibilidade e permite checagem estática (mypy, pyright).

```

1 def calcular_idade_nascimento(ano_atual: int, idade: int) -> int:
2     """Retorna o ano de nascimento."""
3     return ano_atual - idade
4

```

```
5 def buscar_usuario(id: int) -> dict[str, str | int] | None:
6     # ... implementacao ...
7     return None
```

Link principal: **PEP 484 – Type Hints** (<https://peps.python.org/pep-0484/>).

## Docstrings – documente sempre!

Toda função deve ter uma *docstring* (string tripla logo abaixo da def). Siga a **PEP 257 – Docstring Conventions** para padronizar.

Exemplos bons:

- Uma linha para funções simples
- Multi-linha com Args, Returns, Raises (para funções complexas)

Lambda não suporta docstring personalizada — use só para funções curtas e óbvias.

Link PEP 257: <https://peps.python.org/pep-0257/>

---

## Problema

**A.15** Escreva uma função `eh_par(n: int) -> bool` que retorne `True` se o número for par.

**A.16** Crie uma função `contar_vogais(texto: str) -> int` que conte vogais (a,e,i,o,u), ignorando maiúsculas/minúsculas.

**A.17** Escreva `media_notas(notas: list[float]) -> float` que calcule a média. Se a lista estiver vazia, retorne 0.0.

**A.18** Crie uma função `formatar_produto(prod: dict) -> str` que receba um dicionário como `{"nome": "teclado", "preco": 189.90, "estoque": 42}` e retorne: “O produto teclado

custa R\$189.90 e temos 42 unidades em estoque.”

**A.19** (Bônus) Escreva uma função com `**kwargs` chamada `registrar\_log` que imprima “Log: [mensagem]” e todos os `kwargs` extras como `chave=valor`.

**A.20** Adicione `type hints` e `docstring` (estilo PEP 257) à função `saudacao` do exemplo acima.

## Estruturas de Dados

### Listas

Uma lista (`list`) em Python é criada quando utilizamos colchetes (quadrados, `[ ]`) e separamos seus valores por vírgulas. Os elementos de uma lista não precisam ser do mesmo tipo. O acesso às entradas em uma lista ocorrem por meio de operações de indexação ou fatiamento.

```
1 >>> my_list = ["Ola", 93.8, "mundo", 10]
2 >>> my_list[0]
3     <<'Ola'>>
4 >>> my_list[-2]
5     <<'mundo'>>
6 >>> my_list[:2]
7     <<['Ola', 93.8]>>
```

As listas aceitam os seguintes métodos: `append()`, `insert()`, `remove()`, `extend()`, e `pop()`.

```
1 >>> my_list = [1, 2]                # Create a simple list of two integers.
2 >>> my_list.append(4)               # Append the integer 4 to the end.
3 >>> my_list.insert(2, 3)            # Insert 3 at location 2.
4 >>> my_list
5     [1, 2, 3, 4]
6 >>> my_list.remove(3)               # Remove 3 from the list.
7 >>> my_list.pop()                  # Remove (and return) the last entry.
8     4
9 >>> my_list
```

```
10     [1, 2]
11 >>> my_list.extend([5, 6])           # Append multiple values at once.
12 >>> my_list
13     [1, 2, 5, 6]
```

O fatiamento também é muito útil para substituir valores em uma lista.

```
1 >>> my_list = [10, 20, 30, 40, 50]
2 >>> my_list[0] = -1
3 >>> my_list[3:] = [8, 9]
4 >>> print(my_list)
5     [-1, 20, 30, 8, 9]
```

O operador `in` verifica rapidamente se um determinado valor está em uma lista (ou outro iterável, incluindo strings).

```
1 >>> my_list = [1, 2, 3, 4, 5]
2 >>> 2 in my_list
3     <<True>>
4 >>> 6 in my_list
5     <<False>>
6 >>> 'a' in "xylophone"               # 'in' also works on strings.
7     <<False>>
```

## Tuplas

Tuplas (`tuple`) são sequências **imutáveis** de elementos heterogêneos. Elas são criadas principalmente pela **vírgula** (o operador de packing), e **os parênteses são opcionais** na maioria dos contextos — isso é uma característica antiga e fundamental da sintaxe do Python (documentada desde as primeiras versões e reforçada na referência oficial).

```
1 # Parênteses opcionais --- o que define a tupla e a vírgula!
2 coordenadas = 10, 20, 30           # tupla de 3 elementos (sem parênteses)
3 ponto = (3, 4)                     # com parênteses (estilo mais comum e legível)
4 vazia = ()                         # tupla vazia (parênteses obrigatórios aqui)
5 un_elemento = (42,)                # tupla de 1 elemento: vírgula OBRIGATORIA!
6 # un_elemento_errado = (42)         # Isso NÃO é tupla --> é apenas int 42
7
```

```

8 print(type(coordenadas))      # <class 'tuple'>
9 print(coordenadas)           # (10, 20, 30)

```

### Regras práticas para quando os parênteses são necessários ou recomendados:

- **Obrigatórios:** tupla vazia () e tupla de um elemento (x,) — sem a vírgula final, vira apenas um valor solto.
- **Opcionais:** qualquer tupla com 2+ elementos em atribuições, retornos de função, argumentos, etc.
- **Recomendados (PEP 8):** use parênteses para melhorar a legibilidade, especialmente em expressões complexas ou quando a tupla é passada como argumento.

#### NOTA

##### Dica importante (muito comum em iniciantes)

- a, b = 1, 2 → tupla implícita no lado direito (packing)
- x = 5, → tupla de um elemento (sem parênteses!)
- return 1, 2, 3 → retorna tupla automaticamente (muito usado)
- def f(a, b): ... → argumentos são recebidos como tupla se usar \*args

O que **faz** a tupla é a **vírgula**, não os parênteses. Parênteses servem principalmente para agrupar expressões ou aumentar clareza.

### Desempacotamento (unpacking) — funciona com ou sem parênteses:

```

1 x, y, z = 10, 20, 30          # unpacking direto
2 a, b = (5, 10)                # com parenteses (opcional)
3 p, *resto = (1, 2, 3, 4)      # extended unpacking (Python 3+)
4 print(p, resto)              # 1 [2, 3, 4]

```

### Tuplas nomeadas modernas (recomendadas em projetos reais)

Para dados estruturados leves e legíveis, prefira:

- `typing.NamedTuple` (com type hints, desde Python 3.6)
- `dataclasses` (PEP 557, Python 3.7+) — mais flexível, com `frozen=True` para imutabilidade

### Código: Exemplo com `dataclass` imutável

```
1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class Ponto:
5     x: float
6     y: float
7
8 p = Ponto(3.0, 4.0)
9 print(p)                # Ponto(x=3.0, y=4.0)
10 # p.x = 99              # FrozenInstanceError (imutavel!)
```

### Referências:

- PEP 557 – Data Classes (2017, implementado em 3.7)
- Documentação oficial: <https://docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences>

---

## Problema

- A.21** Crie uma tupla com 4 elementos sem usar parênteses e faça unpacking em variáveis `a`, `b`, `*c`.
- A.22** Explique por que `x = 99` resulta em `int` enquanto `x = 99,` resulta em `tuple`. Teste no interpretador.
- A.23** Escreva uma função que retorne 3 valores sem usar parênteses explícitos no `return`.

**A.24** Crie uma `@dataclass(frozen=True)` chamada `Coordenada` com campos `lat: float` e `lon: float`.

**A.25** (Bônus) Use desempacotamento estendido para separar o primeiro, o último e o meio de uma tupla de 7 elementos.

## Dicionários

Dicionários (`dict`) são mapeamentos mutáveis chave-valor, desde Python 3.7 garantidamente ordenados por ordem de inserção (PEP 468 + implementação do CPython).

```

1  pessoa = {
2      "nome": "Ana",
3      "idade": 28,
4      "cidade": "Serra"
5  }
6
7  print(pessoa["nome"])           # Ana
8  pessoa["altura"] = 1.68         # adicao
9  del pessoa["idade"]             # remocao
10
11 # Iteracao (ordem de insercao garantida desde 3.7)
12 for k, v in pessoa.items():
13     print(f"{k}: {v}")

```

Métodos úteis: - `get(chave, default)` — evita `KeyError` - `setdefault(chave, valor)` — insere se não existe - `update(outro_dict)` - `pop(chave)`, `popitem()` (remove e retorna último item desde 3.7)

Desde Python 3.9: - Operadores de merge: `d1 | d2` e `d1 | = d2`

```

1  defaults = {"tema": "escuro", "fonte": "Arial"}
2  config = {"tema": "claro"}
3  final = defaults | config          # {'tema': 'claro', 'fonte': 'Arial'}

```

**NOTA**

Desde Python 3.7 (PEP 468 + CPython detail), dicionários preservam ordem de inserção — não dependa mais de `collections.OrderedDict` para isso.

## Conjuntos (sets)

Conjuntos (set) são coleções **não ordenadas**, **mutáveis** e sem duplicatas. Ideais para testes de pertinência, remoção de duplicatas e operações matemáticas de conjuntos.

```
1 frutas = {"maca", "banana", "laranja"}
2 frutas.add("uva")
3 frutas.remove("banana")           # KeyError se nao existir
4
5 print("maca" in frutas)           # True (muito rapido!)
6
7 a = {1, 2, 3}
8 b = {2, 3, 4}
9 print(a | b)                      # uniao      {1,2,3,4}
10 print(a & b)                     # intersecao {2,3}
11 print(a - b)                    # diferenca   {1}
12 print(a ^ b)                    # diferenca simetrica {1,4}
```

### Conjunto imutável: frozenset

```
1 fs = frozenset([1, 2, 3])
2 # fs.add(4)                        # AttributeError
3 d = {fs: "valor"}                  # pode ser chave de dict
```

**NOTA**

Em 2025–2026, discute-se PEP 814 – frozendict (dicionário imutável built-in), que complementaria frozenset para casos onde `dataclass(frozen=True)` ou `types.MappingProxyType` não são suficientes.

## Problema

- A.26** Crie uma tupla com 5 elementos heterogêneos e faça desempacotamento com `*`.
- A.27** Use um dicionário para contar a frequência de palavras em uma frase (`split` + `loop` + `get`).
- A.28** Remova duplicatas de uma lista usando um `set`, depois converta de volta para lista preservando ordem (dica: `dict.fromkeys` desde 3.7).
- A.29** Crie um `@dataclass` chamado `Produto` com campos `nome: str`, `preco: float`, `estoque: int` = 0, `frozen=True`.
- A.30** (Bônus) Implemente uma função que receba dois conjuntos e retorne se um é subconjunto do outro (`<=`).

## Comandos úteis

A partir do momento que você tem o Python instalado na sua máquina, abrindo o terminal pode digitar `python` e colocar `-V` (traço **V** maiúsculo), como no comando a seguir:

```
python -V
```

obtemos a versão do Python instalada. É muito importante fazermos isso para verificarmos a versão 2 ou 3 do Python. Caso ele mostre que a versão chamada foi a 2, tente o comando:

```
python3 -V
```

a fim de verificar se a versão 3 também se encontra instalada, porém não está sendo instanciada automaticamente pelo sistema operacional. Isso ocorre porque durante a ins-

talação do Python vários binários (executáveis) podem ser instalados com os nomes específicos das versões disponíveis.

Temos ainda o comando `python -c ""`, cujo objetivo é passar um comando para dentro do Python quando não temos um arquivo. Podemos criar um arquivo, e pedimos para o Python executar o seu conteúdo ou fazemos por linha de comando. Ele é útil para automatizar pequenas tarefas e muitas vezes para testarmos algumas ideias. Por exemplo, veja a sequência do comando a seguir:

```
python -c "print(1+5)"
```

Um outro comando muito útil é o `python -m`, que serve para chamarmos módulos do Python. Por exemplo, o Python tem um servidor web embutido. Siga o comando a seguir:

```
python -m http.server
```

ao executar o comando acima, ele importa e dispara o servidor na porta indicada (geralmente a 8000). Abrindo um navegador e digitando no browser `localhost:8000`

## Decorators — encapsulando comportamento

Um *decorator* é uma função que recebe outra função, adiciona comportamento antes e/ou depois dela e retorna a função “modificada” (na prática, um *wrapper*). É uma das funcionalidades mais poderosas e elegantes do Python e será muito usada no Flask (ex.: `@app.route`, `@login\required`, `@admin\only`).

**Código: Decorator: analogia do pao**

```
1 def pao(func):
2     def wrapper(*args, **kwargs):
3         print("--- Fatia superior do pao ---")
4         resultado = func(*args, **kwargs)          # executa o "recheio"
5         print("--- Fatia inferior do pao ---")
6         return resultado
7     return wrapper
```

```

8
9 @pao
10 def hamburguer_vegano():
11     print("Hamburguer vegano no meio!")
12
13 hamburguer_vegano()

```

Saída esperada:

```

--- Fatia superior do pao ---
    Hamburguer vegano no meio!
--- Fatia inferior do pao ---

```

## NOTA

### O que realmente acontece?

- @pao é equivalente a: `hamburguer_vegano = pao(hamburguer_vegano)`
- O decorator recebe a função original como argumento
- Define um *wrapper* que controla a execução
- O código do *func* só roda quando o *wrapper* decidir chamar

### Boa prática: preservar metadados da função original

```

1 from functools import wraps
2
3 def pao(func):
4     @wraps(func)
5     def wrapper(*args, **kwargs):
6         print("Pao superior")
7         r = func(*args, **kwargs)
8         print("Pao inferior")
9         return r
10    return wrapper

```

Sem @wraps, `help(hamburguer_vegano)` mostraria a documentação do *wrapper*, não da função original.

### Exemplo útil para Flask

```
1 def login_required(f):
2     @wraps(f)
3     def decorated_function(*args, **kwargs):
4         if not session.get('logged_in'):
5             return redirect(url_for('login'))
6         return f(*args, **kwargs)
7     return decorated_function
8
9 @app.route('/perfil')
10 @login_required
11 def perfil():
12     return "Bem-vindo ao seu perfil!"
```

## Problema

- A.31** Crie um decorator `tempo_execucao` que imprime o tempo gasto pela função (use `time.perf_counter()`).
- A.32** Escreva um decorator `log` que registra a chamada da função com seus argumentos (ex.: “chamando soma(3, 4)”).
- A.33** (Bônus) Crie um decorator `restrito_admin` que verifica se o usuário é admin antes de executar a função.

## Programação Orientada a Objetos — Classes e Herança

Tudo em Python é um objeto. Uma classe define o molde; a instância é o objeto concreto.

**Código:** Classe `Animal` - exemplo básico

```
1 class Animal:
```

```

2  planeta = "Terra"                # atributo de classe (compartilhado)
3
4  def __init__(self, nome):
5      self.nome = nome
6      self.ja_nasceu = False
7
8  def nascer(self):
9      """Faz o animal nascer."""
10     print(f'Ola! Nasci no planeta {self.planeta}.')
11     self.ja_nasceu = True
12
13  def comer(self):
14     print("Crunch crunch... estou comendo!")

```

### Código: Herança e sobrescrita

```

1  class Mamifero(Animal):
2      def comer(self):
3          print("Estou tomando leite")
4
5  class Reptil(Animal):
6      def nascer(self):
7          print(f"Quebrei o ovo no planeta {self.planeta}!")

```

#### NOTA

##### Métodos especiais (dunder methods)

Métodos com dois underlines (`__init__`, `__str__`, `__len__`, etc.) definem comportamentos especiais. São chamados de “dunder methods”.

### Method Resolution Order (MRO)

Quando há herança múltipla, o Python usa o MRO para decidir a ordem de busca de métodos.

```

1  class Ornitorrinco(Mamifero, Reptil):
2      pass
3

```

```
4 print(Ornitorrinco.__mro__)  
5 # (<class 'Ornitorrinco'>, <class 'Mamifero'>, <class 'Reptil'>, <class 'Animal'>, <  
    class 'object'>)
```

## Problema

- A.34** Crie uma classe `Produto` com atributos `nome`, `preco` e `estoque`. Adicione uma `property` `preco_com_desconto`.
- A.35** Implemente um setter para `preco` que não aceite valores negativos.
- A.36** (Bônus) Crie um mixin `Logavel` com um método `log(mensagem)` que imprime “[LOG] mensagem”.

## IPython

O Python pode ser utilizado de forma interativa por meio de diferentes interfaces. Uma das mais populares e versáteis é o *IPython*<sup>1</sup>, que consiste em um interpretador interativo baseado em console, com recursos avançados que superam os do interpretador padrão da linguagem.

No IPython, as entradas de código são geralmente precedidas por colchetes angulares numerados (veja, por exemplo, In Pérez et. al. (2007) [1]), indicando que o comando será executado imediatamente. Para executar um *script* diretamente no ambiente IPython, utiliza-se o comando `%run`, conforme ilustrado abaixo:

```
1 $ ipython                                # Iniciar o IPython.  
2  
3 In [1]: print("Isso eh simplesmente IPython!")  # Executa o comando.  
4 Isso eh simplesmente IPython!  
5
```

<sup>1</sup>Disponível em: <https://ipython.org/>. Acesso em: 06 ago. 2025.

```

6 In [2]: %run python_intro.py          # Executa um script Python.
7 Ola mundo!

```

Uma das principais vantagens do IPython reside no suporte à introspecção de objetos — funcionalidade que permite visualizar, de forma prática, os atributos e métodos associados a objetos da linguagem, contribuindo para o processo de aprendizagem e depuração de código [3]. Além disso, o ambiente oferece a função integrada `help()`, que facilita o acesso à documentação e promove uma experiência interativa para o usuário.

```

1 # Uma lista eh uma estrutura de dados fundamental em Python.
2 # Para visualizar os metodos associados a uma lista, digite o nome do objeto seguido
  de um ponto e pressione 'tab'.
3 In [1]: list.    # Pressione 'tab'.
4 append()  clear()  copy()    count()  extend()
5 index()   insert() pop()     remove() reverse() sort()
6
7 # Para acessar a documentacao de um metodo especifico, utilize o caractere '?'.
8 In [2]: list.append?
9 Signature: list.append(self, object, /)
10 Docstring: Append object to the end of the list.
11 Type:      method_descriptor
12 # Alternativamente, utilize a funcao 'help()' para acessar ajuda mais extensa:
13 In [3]: help()
14 help> list
15 Help on class list in module builtins:
16 class list(object)
17 | list(iterable=(), /)
18 | ...
19 # Pressione 'q' para sair.
20 help> quit

```

## NOTA

Utilizar o IPython em paralelo a um editor de texto é uma prática recomendada, especialmente durante o aprendizado. Esse ambiente permite testar rapidamente a sintaxe e pequenos blocos de código, o que favorece a identificação precoce de erros e torna o processo de desenvolvimento mais eficiente.

Em caso de dúvidas, recorrer a fontes de apoio como o site `stackoverflow.com` é altamente indicado, dada a sua ampla comunidade e base de conhecimento consolidada.

Por fim, vale lembrar que a melhor forma de aprender uma linguagem de programação é **escrevendo código**. Portanto, pratique os exemplos apresentados ao longo deste material, executando-os diretamente no console do IPython. Evite, sempre que possível, copiar e colar os comandos — ao digitar, você treina não apenas a lógica, mas também a sintaxe da linguagem.

## Lista de Exercícios

**A.1** Converter as seguintes expressões para o interpretador Python:

(a)  $10 + 20 \times 30$

(b)  $4^2 \div 30$

(c)  $(9^4 + 2) \times 6 - 1$

**A.2** No interpretador Python (ou em um arquivo `.py`) resolva a expressão:  $10\%3 \times 10^2 + 1 - 10 \times 4/2$ . Resolva também usando apenas lápis e papel. O resultado a ser encontrado é 81.0

**A.3** Faça um programa que resolva a soma de três variáveis e imprima o resultado na tela.

**A.4** Considerando um salário de R\$ 750.00, determine numericamente um aumento de 15%.

**A.5** Escreva um programa que, a partir das entradas dos valores de dias, horas, minutos e segundos, calcule o tempo total em segundos.

**A.6** Escreva um programa que pergunte três números para o usuário e que devolva como resposta qual o maior deles.

**A.7** Escreva um programa em Python que verifique se um número é primo.

**A.8** O volume de uma esfera com raio  $r$  é  $V = \frac{4}{3}\pi r^3$ . Em seu arquivo Python do problema ??, defina uma função chamada `sphere_volume()` que aceita um único parâmetro  $r$ . Retorne o volume da esfera de raio  $r$ , usando 3,14159 como aproximação para  $\pi$  (por enquanto). Escreva também uma documentação apropriada para sua função.

Para testar sua função, chame-a na cláusula `if __name__ == "__main__"` e imprima o valor retornado. Execute seu arquivo para ver se sua resposta é o que você espera.

**A.9** A função integrada `print()` possui os argumentos de palavras-chave úteis `sep` e `end`. Ele aceita qualquer número de argumentos posicionais e os imprime com `sep` inserido entre valores (padrão para um espaço), então imprime `end` (padrão para o caractere de nova linha `'\\n'`).

Escreva uma função chamada `isolate()` que aceite cinco argumentos. A função deve imprimir os três primeiros argumentos separados por 5 espaços e depois imprimir os dois últimos argumentos com um único espaço separando os três últimos argumentos. Por exemplo,

```
1 >>> isolate(1, 2, 3, 4, 5)
2 1      2      3 4 5
```

**A.10** Escreva duas novas funções, chamadas `first_half()` e `backward()`.

- (a) `first_half()` deve aceitar um parâmetro e retornar a primeira metade dele, excluindo o caractere do meio se houver um número ímpar de caracteres.  
(Dica: a função interna `len()` retorna o comprimento da entrada.)
- (b) A função `backward()` deve aceitar um parâmetro e inverter a ordem de seus caracteres usando fatiamento e, em seguida, retornar o parâmetro invertido.  
(Dica: o parâmetro `step` usado no fatiamento pode ser negativo.)

Use o IPython para testar rapidamente a sintaxe de cada função.

# Referências Bibliográficas

- [1] FOUNDATION, P. S. *PEP 257 - Docstring Conventions*. 2001. <https://peps.python.org/pep-0257/>.
- [2] FOUNDATION, P. S. *Builtin Module \_\_main\_\_ - Python Documentation*. 2025. [https://docs.python.org/3/library/\\_\\_main\\_\\_.html](https://docs.python.org/3/library/__main__.html).
- [3] PÉREZ, F.; GRANGER, B. E. Ipython: a system for interactive scientific computing. *Computing in Science & Engineering*, IEEE, v. 9, n. 3, p. 21–29, 2007.
- [4] ROSSUM, G. V.; WARSAW, B.; COGHLAN, N. *PEP 8 – Style Guide for Python Code*. 2001. <https://peps.python.org/pep-0008/>. Python Enhancement Proposal.